



Patterns behind Chaos: Forecasting Data Movement for Efficient Large-Scale MoE LLM Inference

Zhongkai Yu

UCSD

La Jolla, USA

zhy055@ucsd.edu

Yue Guan

UCSD

La Jolla, USA

y9guan@ucsd.edu

Zihao Yu

Indiana University Bloomington

Bloomington, USA

yuzih@iu.edu

Chenyang Zhou

Columbia University

New York, USA

cz2791@columbia.edu

Zhengding Hu

UCSD

La Jolla, USA

zhh068@ucsd.edu

Shuyi Pei

Samsung Semiconductor

San Jose, USA

shuyi.pei@samsung.com

Yangwook Kang

Samsung Semiconductor

San Jose, USA

yangwook.k@samsung.com

Yufei Ding

UCSD

La Jolla, USA

yufeid@ucsd.edu

Po-An Tsai

NVIDIA

Santa Clara, USA

poant@nvidia.com

Abstract—Large-scale Mixture of Experts (MoE) Large Language Models (LLMs) have recently become the frontier open-weight models, achieving remarkable model capability similar to proprietary ones. But their random expert selection mechanism introduces significant data movement overhead that becomes the dominant bottleneck in multi-unit LLM serving systems.

To understand the patterns underlying this data movement, we conduct comprehensive data-movement-centric profiling across four state-of-the-art large-scale MoE models released in 2025 (200B–1000B) using over 24,000 requests spanning diverse workloads. We perform systematic analysis from both temporal and spatial perspectives and distill six key insights to guide the design of diverse serving systems. We verify these insights on both future wafer-scale GPU architectures and existing GPU systems. On wafer-scale GPUs, lightweight architectural modifications guided by our insights yield a 6.6x average speedup across four 200B–1000B models. On existing GPU systems, our insights drive the design of a prefill-aware expert placement algorithm that achieves up to 1.25x speedup on MoE computation. Our work presents the first comprehensive data-centric analysis of large-scale MoE models together with a concrete design study applying the learned lessons. Our profiling traces are publicly available at https://huggingface.co/datasets/core12345/MoE_expert_selection_trace.

Index Terms—Mixture of Experts, Large Language Model, Wafer-Scale GPU, Profiling, LLM Serving System

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable capabilities across diverse domains, including programming assistance [1], [2], translation [3], [4], and chatbots [5], [6]. Since the beginning of 2025, large-scale Mixture of Experts (MoE) LLMs (200B+ model with 100+ experts) have become the leading models for frontier LLMs [7] and the most widely used open weight models.

Unlike dense LLMs that activate all model weights uniformly, MoE models dynamically route each token to only a subset of experts, introducing substantial data movement overhead. Such overhead already exceeds 50% of execution time for small models (e.g., Mixtral 8x7B) on modest systems (2–4 GPUs), and it exacerbates further with larger models such as DeepSeek V3 with 32× experts and 15× parameters deployed on multi-node systems (32+ GPUs) [8], [9]. Moreover, this

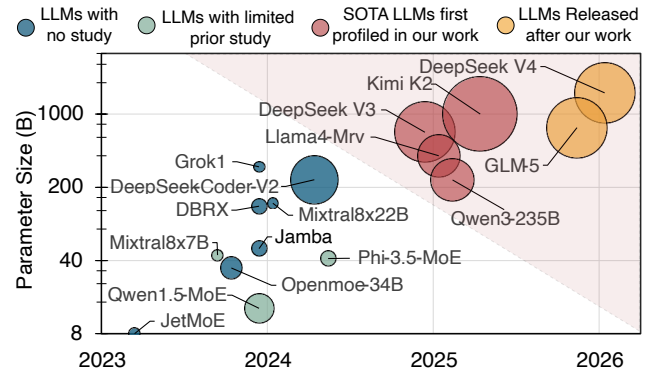


Figure 1. MoE LLM models sizes and release dates. Bubble size indicates the number of experts in each layer. Prior studies [13], [15]–[17] provide limited analysis of smaller models from narrow perspectives, while our work presents the first comprehensive analysis of multiple unstudied SOTA models.

scaling trend is accelerating: recent releases such as DeepSeek V4 [10] and GLM-5 [11] continue to push the frontier, making the associated data movement patterns ever more critical. Yet as shown in Figure 1, no prior work has systematically investigated these patterns at scale. Earlier studies [12]–[14] confined themselves to profiling one or two small MoEs on limited hardware, reporting surface-level observations without system-level insights. As parameter sizes and expert counts surge, new data movement patterns have emerged but remain unexplored, leaving significant optimization opportunities on the table. **A comprehensive characterization of data movement in SOTA MoE models therefore presents a fruitful opportunity for better efficiency.**

If data movement in MoE models were fully unpredictable, it would present significant challenges for deployments on multi-unit systems. **From a temporal perspective**, the explosive growth in expert combinations would make it impossible to prefetch, cache, or replicate experts in advance. For example, large-scale MoE models like DeepSeek V3 have

$C_{256}^8 = 4,426,165,368$ combinations in expert selection. When served with host memory-offload systems, such unpredictability would result in data movement like expert migrations between GPU and host, incurring substantial overhead, as inter-unit communication becomes the primary bottleneck. **From a spatial perspective**, if expert selection were truly random, it would lead to severe workload imbalance across units. When queries from diverse tasks are served concurrently, the number of queries assigned to each expert would vary dramatically, creating significant workload disparities. Consequently, most units would remain idle and wait for heavily loaded units to finish, resulting in poor hardware resource utilization.

Fortunately, as we later show in the paper, **MoE expert selections indeed have predictability that designers can exploit to reduce data movement**. To uncover the inherent patterns in MoE models, we conduct a comprehensive data-movement-centric profiling of four state-of-the-art MoE models ranging from 235B to 1000B parameters released in 2025. As highlighted in Figure 1, we profile DeepSeek V3 [18], Llama4 Maverick [19], Qwen3-235B [20], and Kimi K2 [21] across 24,000 requests involving varied tasks, topics, and languages, which consumes >2000 GPU hours in total. We then collect the expert selection trace of all layers and tokens in each request to create an expert selection database of over 150 GB JSON files. From these extensive traces, we conduct a comprehensive analysis to uncover data movement patterns from both temporal and spatial perspectives, making our findings *system-agnostic* and applicable to various serving architectures at any scale. We then distill six key insights that serve as a solid foundation to understand MoE data movement and directly inform future MoE LLM serving system design, addressing critical questions that have remained unanswered in the field, such as: *Is there a correlation between previously selected experts and those selected later? Are there discernible rules underlying the observed expert selection skewness? Do different tasks tend to activate different experts?* Our work represents the first systematic effort to characterize data movement patterns at the scale of up-to 1000B model across a wide range of tasks, providing actionable insights that can guide the design of next-generation MoE serving systems.

To demonstrate the broad applicability of our insights, we present case studies on both future and existing GPU systems. On the architecture side, we observe that modern GPUs have already adopted multi-chiplet designs due to single-die size limitations [22]–[24] and are evolving toward wafer-scale integration enabled by emerging on-wafer packaging technologies [25], [26]. Targeting this trend, we develop a two-level data-placement-aware command processor and a hardware-managed HBM scheme that jointly balance workload across dies and reduce inter-die communication, achieving an average $6.6\times$ speedup in MoE serving throughput on wafer-scale GPUs. On existing multi-GPU systems, we observe that prefill-stage expert selections can effectively predict decode-stage behavior. Building on this observation, we propose prefill-aware expert placement algorithms to reduce decode workload imbalance, and achieve up to $1.25\times$ speedup. Our main

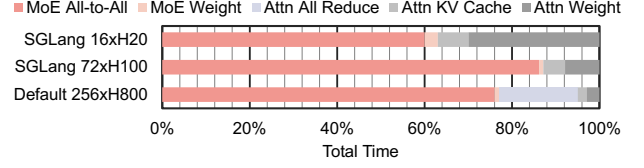


Figure 2. Latency breakdown for different data movement in DeepSeekV3 (4K sequence), modeled after various serving configurations [18], [27], [28].

contributions can be summarized as follows:

- We propose a comprehensive and systematic data-movement-centric profiling across four latest, large-scale MoE models released in 2025 between 235B and 1000B to uncover the data movement patterns from both temporal and spatial perspectives.
- We distill six key insights for designing efficient MoE serving systems based on our profiling and analysis, providing actionable guidance that can inspire future research in MoE serving systems.
- Leveraging these insights, we conduct case studies on both future and existing GPU systems. On future wafer-scale GPUs, we improve MoE throughput by $6.6\times$ with minor hardware modifications. On existing multi-GPU systems, we achieve up to $1.25\times$ speedup on an $8\times H100$.
- We collect over 70,000 expert selection traces across multiple models and datasets, totaling over 150 GB in JSON format, and have open-sourced all traces with our multi-chiplet simulator to facilitate future research.

II. BACKGROUND

A. LLM and MoE Model Architecture

Most state-of-the-art LLMs adopt a decoder-only transformer architecture that follows a token-by-token autoregressive workflow [29]. As shown in Figure 3(a), after users input queries, the serving process is divided into two stages: the prefill stage and the decode stage. During the prefill stage, all input tokens are processed simultaneously to generate the first output token. The decode stage follows immediately, where tokens are generated sequentially. The generated token from each iteration is appended to the input sequence to produce the next token in the following iteration.

The Mixture of Experts (MoE) mechanism is a state-of-the-art approach to improve LLM performance and has become prevalent among current frontier LLMs [30]. As shown in Figure 3(b), MoE-based LLMs replace the feed-forward network (FFN) layers in traditional LLMs with MoE layers. In each layer, multiple experts are deployed, and each request is routed to a small subset of the most suitable experts based on a gating mechanism. This innovation enables MoE models to scale model parameters without incurring extra inference overhead, since only a fraction of parameters are activated for each request. However, this mechanism also introduces dynamic randomness, since expert selection is unknown until gating is completed, posing new challenges for serving systems.

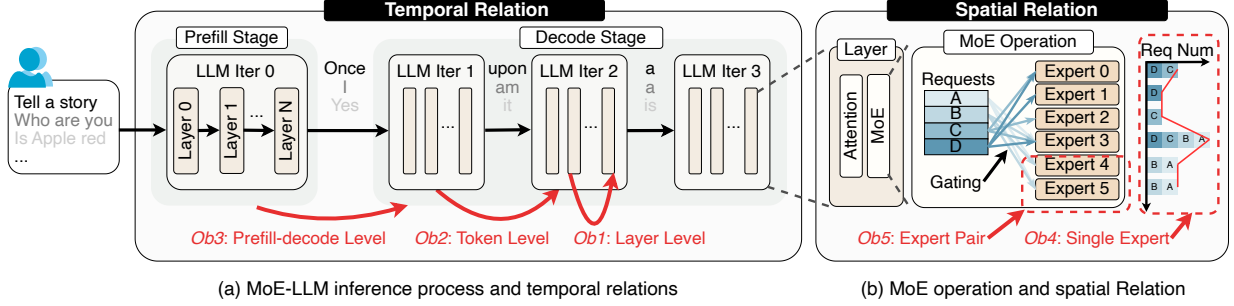


Figure 3. Inference process of MoE LLMs and the categorization method for our proposed data-centric profiling approach.

B. Prior MoE Serving Systems

The MoE mechanism constitutes the primary source of data movement overhead in modern serving systems. As illustrated in Figure 2, take DeepSeek V3 as an example, MoE-related data movement (MoE All-to-All and MoE Weights) dominates the overhead across different serving configurations, accounting for 60%-90% of total latency under 4K sequence length. To address this, existing research has developed numerous system-level solutions targeting different performance and cost objectives. Edge systems like MoE-Lightning [31] and CoServe [32] employ CPU memory offloading techniques to address GPU memory capacity constraints, while cloud systems such as Comet [9] and MegaScale-Infer [15] target multi-GPU systems and address GPU-GPU communications in MoE for higher throughput. Novel hardware architectures like Duplex [33] explore processing-in-memory to accelerate data movement in MoE LLMs.

However, these prior studies employ a *system-centric* methodology when optimizing for MoE LLMs. Namely, they inherently focus on a specific platform and the corresponding data movement patterns of MoE in such platform (e.g., CPU-GPU, multi-GPU, ML accelerators). As a result, they propose deployment-specific optimizations that may not generalize across different serving platforms, and their insights are often a slice of the overall inherent patterns in MoE LLMs.

In this work, we flip the process and adopt a *model-centric* strategy by conducting system-independent profiling to extract *system-agnostic* insights about MoE data movement patterns. These insights are therefore broadly applicable across various platforms, providing a foundation for optimization strategies that transcend specific system implementations.

III. MOE PROFILING AND SYSTEM INSIGHTS

In this section, we conduct a data-movement-centric profiling of the expert selection behavior in four state-of-the-art MoE models: Deepseek V3 (671B), Llama4-Maverick-128E (402B), Qwen3-235B (235B), and Kimi K2 (1000B). All results are averaged over more than 24,000 requests.

A. Categorization Methodology

As in Figure 3, we categorize MoE expert selection profiling results into two categories: *temporal* and *spatial* relations.

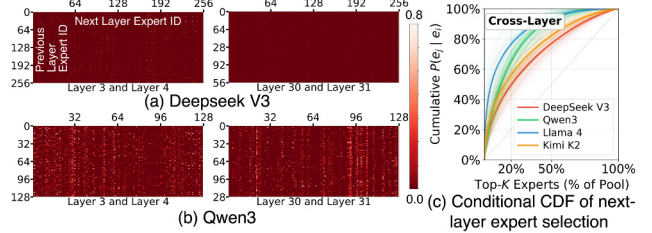


Figure 4. Cross-layer expert correlation. (a, b) Joint co-activation heatmaps between layers N and $N+1$ in DeepSeek-V3 and Qwen3. (c) Conditional CDF $P(e_j | e_i)$ for each layer's top-1 expert.

Temporal relations capture time-dependent expert selection patterns where current choices inform future selections. These patterns enable *single-unit strategies* that optimize data movement for individual units through prefetching, caching, and data migration. For example, in multi-chiplet GPU systems, caching experts in local DRAM after remote fetches significantly reduces inter-unit communication. To exploit temporal predictability, we analyze expert selection at multiple time scales shown in Figure 3(a): *layer-level*, *token-level*, and *prefill-decode-level* patterns.

Spatial relations capture how expert activations are distributed across compute units within a given time window. This distributional information enables *multi-unit strategies* that optimize expert placement and workload balancing across the system, reducing data movement and preventing bottlenecks. We classify spatial relations into *single-expert activation imbalance* and *expert-pair co-activation affinity* as shown in Figure 3(b), and investigate how task types influence these patterns to inform system-level optimization.

B. Temporal Relations

As shown in Figure 3(a), we classify the temporal relations of expert selection into three categories, arranged in order of increasing time scale. At the layer level, we examine the relationship between two adjacent model layers. At the token level, we focus on the same model layer across two adjacent tokens. At the stage level, we analyze the relationship between the prefill stage and the decode stage.

1) Layer-Level Correlation: (Ob1)

As shown in Figure 4 (a) and (b), we present heatmaps for

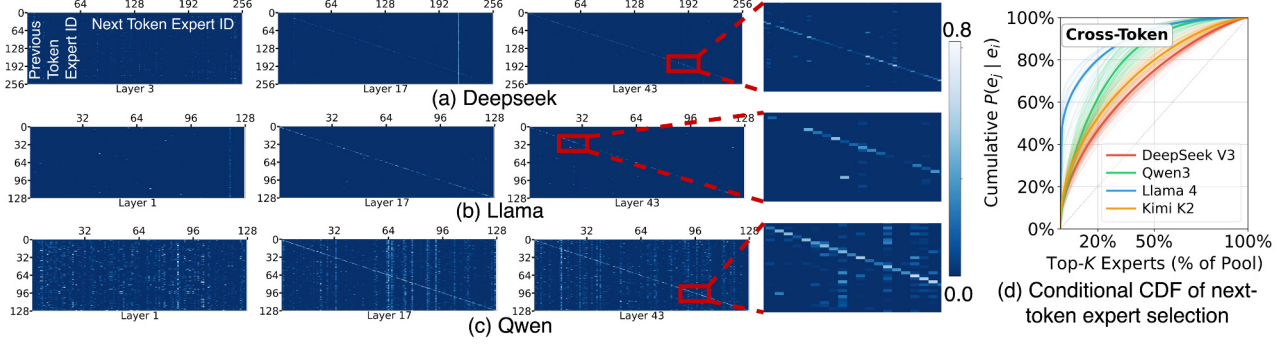


Figure 5. Cross-token expert correlation. (a, b, c) Joint co-activation heatmaps between tokens t and $t+1$ in DeepSeek-V3, Llama 4, and Qwen3. (d) Conditional CDF for each layer’s top-1 expert: top 20% of the next-token expert already covers most of probability mass.

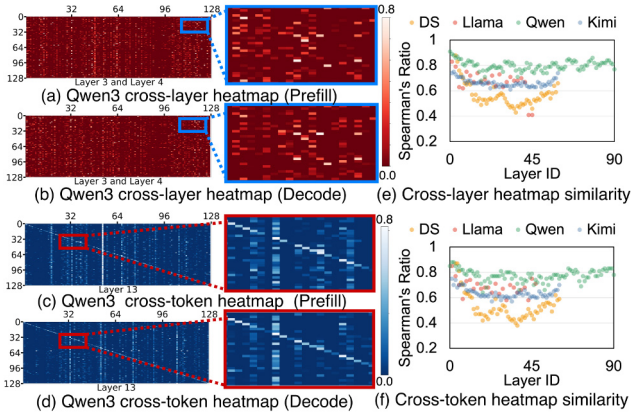


Figure 6. Expert activation patterns remain consistent across prefill and decode stages for both (a, b) cross-layer heatmap and (c, d) cross-token heatmaps. Spearman’s ratio quantified in (e, f) shows a strong relation (≥ 0.7).

Deepseek and Qwen illustrating expert selection relationships across adjacent layers. Each pixel in the heatmap displays the conditional probability of selecting expert j in the next layer given that expert i was activated in the previous layer, with bright colors indicating higher probabilities.

The heatmaps reveal clear cross-layer correlations with white dots highlighting specific expert pairs with significantly higher selection probabilities across adjacent layers. However, correlation patterns vary across layers within the same model and differ between models due to architectural variations. For instance, patterns between layers 3-4 differ from those between layers 30-31. Qwen3’s notably brighter heatmap indicates stronger cross-layer correlations than Deepseek. Beyond the white dots, there are also consistent bright vertical lines, suggesting certain experts are frequently chosen regardless of previous layer selections. These patterns indicate generally popular experts, analyzed further in Sec. III-C1.

To quantify these relationships, we analyze the conditional CDF $P(e_j | e_i)$ in Figure 4(c): the top 20% of next-layer candidates already cover 50%, 65%, 77%, and 56% of the conditional probability mass for DeepSeek-V3, Qwen3,

Llama 4¹, and Kimi K2, respectively. This reveals strong, model-dependent cross-layer correlations, with Llama4 showing the strongest effect and Deepseek the weakest.

2) Token-Level Correlation: (Ob2)

We examine expert selection relations for the same layer between adjacent tokens in Figure 5. Each pixel in the heatmap displays the conditional probability of selecting expert j in the next token given that expert i was activated in the previous token, with bright colors indicating higher probabilities.

Similar to layer-level patterns, cross-token heatmaps exhibit white dots, bright vertical lines, and variation across layers and models, indicating correlations between adjacent tokens. However, token-level relations reveal a common pattern appearing across all models: the bright diagonal line that indicates the tendency to select the same expert across adjacent tokens. This diagonal pattern emerges predominantly in higher layers (17 and 43) but not in lower layers (1 and 3), regardless of models.

We apply the same conditional-CDF analysis to the token-level relation. As shown in Figure 5(d), the top 20% of next-token expert candidates cover 47%, 62%, 80%, and 53% of the cumulative conditional probability in DeepSeek-V3, Qwen3, Llama 4, and Kimi K2, respectively, averaged across all MoE layers. The correlation is again strongest in Llama 4 and weakest in DeepSeek-V3.

3) Prefill-Decode-Level Correlation: (Ob3)

Building on the layer-level and token-level relations, we observe notable similarities in expert selection patterns between prefill and decode stages. Comparing heatmaps across stages in Figure 6(a)(b)(c)(d), we find similar distributions of bright dots, indicating that expert pair heatmap during prefill and decode shares similarities. This cross-stage consistency suggests us that the prefill-collected information can guide initial decode steps until sufficient decode data accumulates.

To quantify this similarity, we compute Spearman’s ratio (ρ) across all model layers, comparing prefill and decode heatmaps. Spearman’s Ratio ρ measures monotonic relationships between variables, ranging from -1 (perfect negative correlation) to 1 (perfect positive correlation). Generally, $|\rho| >$

¹Llama 4 inserts dense FFN layers between MoE layers, so we pair adjacent MoE layers (N and $N+2$).

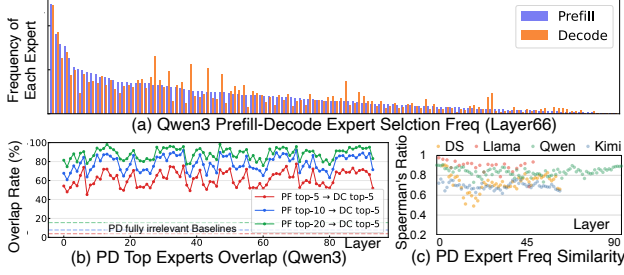


Figure 7. (a) Expert frequency distributions between prefill and decode stages exhibit similarity. (b) The most popular experts in the two stages overlap substantially. (c) All models show a high Spearman correlation between prefill and decode expert frequencies.

0.7 indicates strong correlation, $0.4 < |\rho| \leq 0.7$ indicates moderate correlation, and $|\rho| \leq 0.4$ suggests weak correlation [34]. The results in Figure 6(e)(f) show that most layers demonstrate strong correlation, while a few show moderate correlation. This makes it possible to predict decode-stage expert selection with prefill-stage data.

Beyond expert-pair heatmaps, we also identify prefill-to-decode correlation at the single-expert frequency level. As shown in Figure 7(a), the frequency distributions of prefill and decode stages are substantially similar, though some discrepancies exist among low-frequency experts. To examine the most popular experts, we report the overlap rate of top experts between stages in Figure 7(b): the top-5 prefill experts cover around 60% of the top-5 decode experts, rising to 75% and 90% for top-10 and top-20, respectively. This indicates that prefill information can help predict the hottest decode experts. The cross-model Spearman correlation in Figure 7(c) confirms this relationship holds across all four models.

4) **System Insights from Temporal Relation:** The observed temporal relations in expert selection motivate us to design fine-grained, dynamic strategies on every single unit to reduce data movement. For example, when expert weights are read from remote memory, such as remote DRAM in multi-chiplet systems, or CXL extension memory in memory-disaggregated systems, caching, migration, and prefetching strategies can be deployed to reduce data movement.

★**Insight 1: Prefill-data-driven prediction (Ob3).** Leverage the expert selection trace from the prefill stage to predict expert selection during the decoding stage.

Empirical analysis shows that expert selection patterns during prefill exhibit strong similarity to those during decode. Thus, expert selection information collected in the prefill phase can serve as a valuable reference for predicting decode-phase selections, particularly at the beginning of decoding when only a few tokens have been generated and historical context is scarce. Our section VI demonstrates how prefill information can guide expert placement during decode. This is especially relevant in modern PD-disaggregated serving systems, where

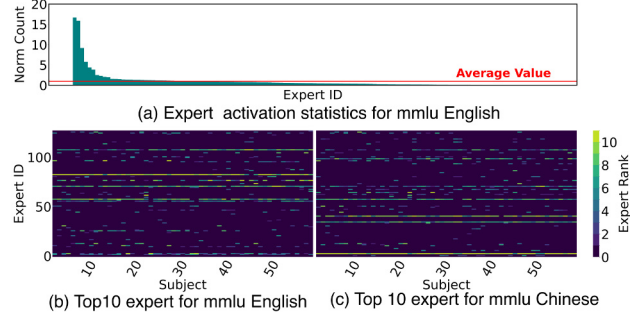


Figure 8. Single-expert spatial relation analysis of Llama4 layer 7 shows: (a) non-uniform expert activation distribution; (b) expert selection strongly correlates with task type; (c) expert activation patterns shift significantly when language changes while content remains identical.

the prefill and decode stages execute on separate machines.

★**Insight 2: Cross-hierarchy memory management (Ob1, Ob2).** Token- and layer-level temporal relations enable dynamic expert prefetching and caching across memory hierarchies.

Layer-level and token-level temporal relations are similar in definition but differ in reuse distance, making them suitable for different levels of the memory hierarchy. Layer-level relations exhibit short reuse distances because consecutive MoE layers execute in immediate succession, while token-level relations incur longer reuse distances because a new token is generated only after traversing all layers.

This maps naturally onto the multi-level memory hierarchies in modern serving systems. For example, in multi-chiplet architectures, each die contains both an LLC and local DRAM, forming a two-tier hierarchy. The faster but smaller LLC is well-suited to managing experts with short reuse distances (layer-level), while the larger local DRAM accommodates experts with longer reuse distances (token-level). Accordingly, we can leverage layer-level relations for LLC management and token-level relations for DRAM management.

This principle generalizes to other system configurations: CXL-based systems with local DRAM and remote CXL memory, SSD offloading systems with DRAM and flash storage, and PIM systems with local and remote DRAM dies. In each case, layer-level relations guide the faster memory tier and token-level relations guide the slower one.

C. Spatial Relation

As shown in Figure 3(b), we analyze spatial patterns in expert selection for both single-expert activation imbalance and expert pair co-activation affinity. For single experts, we examine statistical skewness and the factors affecting each expert’s activation. For expert pairs, we analyze co-activation properties across all two-expert combinations.

1) Single Expert Activation Imbalance: (Ob4)

We examine expert selection frequency at each layer, presenting results for layer 7 of Llama4 in Figure 8. We observe

pronounced skewness where a subset of experts is activated over 16 times more frequently than average. This workload imbalance suggests system designs should duplicate or decentralize frequently used experts.

To investigate selection patterns across different tasks, we analyze all 57 MMLU subjects spanning diverse fields, including biology, history, and math, etc [35]. Figure 8(b) shows the top 10 most popular experts for each subject. Horizontal bright lines indicate certain experts are consistently activated regardless of subject, while remaining popular experts vary significantly between subjects, demonstrating both overlap and distinction in task-based expert selection.

We further examine task impact using the Chinese version of MMLU in MMLU Pro [36] with identical questions but different languages. Figure 8(c) reveals distinctly different patterns: although 5-6 experts remain popular across subjects, only two overlap with English MMLU’s most frequently selected experts. This confirms that task characteristics, including language, significantly influence expert selection, enabling task-aware serving systems that optimize expert distribution to balance workloads and reduce data movement.

2) Expert Pair Co-activation Affinity: (Ob5)

Beyond single expert patterns, we observe spatial relations for expert pairs where certain experts are more likely to be co-activated. We present co-activation heatmaps in Figure 9(a)(b), where both axes indicate expert IDs. Each pixel represents an expert pair with values showing co-activation frequency normalized by theoretical random selection probability: $p = \frac{2}{n(n-1)}$, where n is the number of experts.

Bright dots appear with probabilities 20-40 times higher than theoretical values, indicating strong co-activation tendencies. All heatmaps exhibit central symmetry since expert pair (i, j) equals (j, i) . In Deepseek’s heatmap Figure 9(a), frequently activated pairs lie between red lines forming bright squares, reflecting Deepseek’s routing restriction where tokens are routed only to adjacent nodes to reduce communication overhead. This suggests the potential of separating co-activated expert pairs to balance workload.

We quantify this relation by in Figure 9(c). The top 10% of expert pairs account for 60-80% of total activations, indicating strong skewness. This suggests the potential for separating co-activated expert pairs to balance the workload. We only analyze Deepseek and Qwen since Llama selects one expert per MoE layer, eliminating co-activation relations.

3) **System Insights from Spatial Relation:** Spatial Relation enables coarse-grained, static strategies to address workload imbalance across the system. These strategies could be applied at system startup or during periodic redistribution (e.g., every 10 minutes) through appropriate task distribution.

★**Insight 3: Expert-placement-aware workload distribution (Ob4, Ob5).** Employ expert placement information to design workload-balanced task distribution strategies.

Expert placement in serving systems can change dynam-

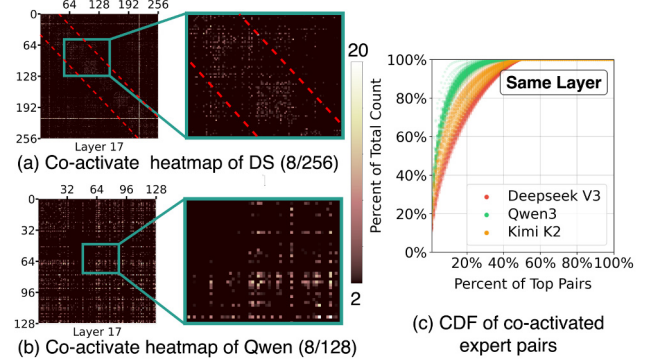


Figure 9. Expert-pair co-activation affinity. (a)(b) Heatmaps for DeepSeek and Qwen. (c) CDF of co-activated expert pairs across all layers: a small fraction of expert pairs accounts for the majority of co-activations.

ically due to expert migration strategies. Therefore, when allocating workload to system units, expert placement should be considered for better workload balance. Besides, the design space for task allocation could be enlarged with emerging new systems. Traditional multi-GPU systems tend to allocate experts to local GPUs to avoid cross-unit communication. However, in multi-chiplet GPUs, we can consider allocating tasks to remote dies for better workload balance as inter-unit communication becomes faster.

★**Insight 4: Popular expert decentralization (Ob4).** Duplicate or decentralize frequently used experts to balance workloads.

Expert skewness causes workload imbalance and suboptimal resource utilization. Duplicating popular experts across multiple compute units distributes load more evenly. Additionally, avoiding co-location of highly popular experts in the same unit further enhances workload balance.

★**Insight 5: Expert-pair separation (Ob5).** Separate frequently co-activated expert pairs to maximize parallelism.

Certain experts are frequently activated simultaneously, exhibiting strong co-activation patterns. Assigning these co-activated expert pairs to different compute units maximizes hardware parallelism and prevents workload concentration on specific units. However, separation also introduces cross-unit communication overhead. The effectiveness depends on system topology and batch size, requiring careful trade-off between parallelism benefits and communication costs.

★**Insight 6: Workload-aware serving system (Ob4).** Leverage the workload information, like task type and language, to make expert migration prior to serving.

Hot experts vary by task and language. English queries, for instance, activate different expert subsets than Chinese queries.

Providing task metadata during serving enables proactive expert placement: when workloads are predominantly English, systems can pre-duplicate or reassign English-relevant experts, reducing communication and balancing loads. This task-to-expert mapping requires only one-time offline profiling per model and can be reused throughout deployment, making the approach practical and efficient.

IV. CASE STUDY 1: WAFER-SCALE GPU ARCHITECTURE DESIGN FOR MOE SERVING

In this section, we adopt future GPU architecture design as a use case to validate our proposed insights. We follow [Insight 3](#) to design a task allocation algorithm and leverage the temporal relation insights ([Insight 1](#) and [Insight 2](#)) to build a data-driven predictor. We also make slight architectural modifications to support the proposed strategies.

A. Trend of Future GPU Architecture

GPU vendors are increasingly adopting multi-chiplet architectures to overcome single-die performance limitations. As Moore’s Law approaches its limits [37] and single-die size remains constrained by photomask dimensions (800-1,000 mm²), advanced packaging technologies like TSMC’s CoWoS [38], Samsung’s X-Cube [39], and Intel’s EMIB [40] enable multiple chiplets within a single package. Leading vendors have adopted such designs: AMD’s MI300 [22] integrates eight compute chiplets, NVIDIA’s Blackwell features two chiplets [24], and upcoming Rubin expects four [41].

This trend is evolving toward wafer-scale systems [42]. TSMC’s System-on-Wafer (SoW) technology accommodates up to 24 compute dies and 96 HBM dies on a single wafer, exceeding 200,000 mm² [43]. As shown in Figure 10(a), a typical wafer-scale multi-chiplet GPU consists of multiple units, each containing a GPU die and several HBM dies interconnected in a mesh topology. Such systems contain over 3 TB of HBM and PFLOPS-level computing power, supporting extremely large models and batch sizes.

The TSMC SoW technology shown in Figure 10(b) connects each GPU die to local HBM dies through local-silicon interconnects (LSI) [44], with adjacent GPU dies communicating via LSI vertically or XSR SerDes links horizontally. Although LSI and SerDes both provide terabit-level bandwidth, inter-GPU-die communication remains the primary bottleneck. Remote data access requires multiple hops across die-to-die links, resulting in high latency. Simultaneous remote HBM access by multiple dies creates bandwidth contention and traffic congestion, further degrading performance.

B. Background on Wafer-scale GPU Programming model

The programming model of future wafer-scale chip remains an open question, but two major candidates have emerged: the multi-GPU-like and the single-GPU-like programming model.

Multi-GPU-like programming model. WSC-LLM [26] and MoEntwine [45] adopt a multi-GPU-like programming model that exposes the entire wafer as a multi-GPU system.

Programmers can program the wafer similarly to a conventional multi-GPU system, with the key difference being the 2D mesh topology where each die communicates directly only with its neighbors. While this approach offers fine-grained control over individual dies and enables flexible software strategies, it diverges from current industry trends. For instance, although Blackwell and Rubin both integrate two compute dies, NVIDIA exposes no toolchain for fine-grained die-level control. Multi-Instance GPU (MIG) can partition a multi-die GPU into independent GPU instances, making each die act independently, but the high-speed D2D link is disabled in this mode [46], forcing inter-die communication through the 10-100× slower NVLink or PCIe and negating the benefit of multi-die packaging. Therefore, extending this programming model to wafer-scale GPUs would require substantial architectural changes to current GPU designs, including redesigning the D2D/C2C controller workflow and distributed LLC structure, making it infeasible in the near term.

Single-GPU-like programming model. HDPAT [47], Hecaton [48], and our work adopt a single-GPU-like programming model that exposes the entire wafer as a unified GPU, fully abstracting the multi-die topology and data placement from software so that the programming experience is identical to that of a monolithic GPU. We adopt this model for two key reasons. First, it aligns with commercial multi-chiplet GPUs such as Blackwell and Rubin, ensuring practical industry relevance. Second, it eliminates the programming complexity of explicit inter-die communication management through libraries such as NCCL or NVSHMEM. However, this programming simplicity shifts the optimization burden entirely to the hardware. Given the inherently distributed architecture, local versus remote data access costs vary by up to 15×, yet the abstraction prevents programmers from controlling cross-die data movement. Consequently, architecture-level optimizations become critical to achieve high hardware utilization.

C. Challenges of serving MoE with future GPUs

Unlike current multi-GPU systems, wafer-scale GPUs can fit entire MoE models on a single chip and support batch sizes over 10,000. However, current GPU architectures introduce two key limitations for such large-scale chips.

Simplistic Task Allocation. Current GPUs integrate a CPU in their SoC to serve as a command processor and allocate tasks to all SMs. However, the traditional command processors treat all SMs equally, ignoring their physical locations and data placement [49], [50]. This oblivious task-to-SM assignment generates excessive D2D traffic and ignores MoE expert selection skewness, leading to poor utilization when most dies remain idle while others become overloaded.

Inadequate Local HBM Management. Current GPUs treat all HBM dies as uniform memory space, but wafer-scale GPUs connect each compute die directly to local HBM, where access is significantly faster than a remote HBM. Frequently accessed experts in remote HBM could be cached locally to minimize D2D traffic, but current GPUs do not distinguish between local and remote HBM and therefore generate unnecessary traffic.

Algorithm 1: Task Allocation Algorithm

Input: *expert_reqs_dict*, *expert_die_map*
Output: *allo_plan*

```

1 Initialize the workload of each die: load_per_die;
2 Sort (expert_reqs_dict, by req_num ascending);
3 for (expert_id, req_num) in expert_reqs_dict do
4   candi_list  $\leftarrow$  GenCandidateList (expert_id, dis=1);
5   candi_list  $\leftarrow$  Sort (candi_list,  $i \mapsto$  load_per_die[i])
   while req_num > 0 do
6     costs_of_dies  $\leftarrow$  CostModel (candi_list);
7     target_die  $\leftarrow$  Argmin (costs_of_dies);
8     allo_plan.append (expert_id, target_die, req_blk);
9     Update (load_per_die);
10    req_num  $\leftarrow$  req_num - req_blk;
11  allo_plan  $\leftarrow$  MergeTasks (allo_plan);
12 return allo_plan;
13 Function GenCandidateList (expert_id, dis):
14   local_die_list = expert_die_map[expert_id];
15   remote_die_list = FindNearDies (local_die_list, dis);
16   return local_die_list + remote_die_list;

```

updates the address into ATU, and sets the *is_local* bit in PDU’s Prediction Table to 1.

Local Data Read (Duplicated): When an SM reads remote data already cached in local HBM (❶), the ATU translates the remote address to a local address and redirects the request to LLC (❷). The LLC and memory controller then retrieve data and send it back to the SM (❸, ❹).

5) *Algorithm Design:* This subsection presents our task allocation algorithm and data-driven predictor, both implemented by the global CP.

Task Allocation Algorithm: Since accurate task distribution is NP-hard, we propose two heuristic mechanisms: a candidate mechanism that reduces the number of dies to consider and a block-granularity distribution mechanism that searches for approximate solutions among candidates.

This algorithm splits MoE kernel computation into sub-tasks for individual dies based on expert selection and distribution information. As shown in algorithm 1, the input *expert_reqs_dict* contains the number of requests belonging to each expert, while *expert_die_map* provides dynamic expert distribution information from the Expert Distribution Table in Figure 10(c), indicating where each expert is stored.

The algorithm iterates through all experts to generate allocation plans. For each expert, it creates a candidate die list including dies storing expert weights and their adjacent dies (shown as green and red in Figure 11(a)). We sort candidates by workload and limit the list to *max_split_num* dies, determined by the expert’s request count (line 3-5). Requests are distributed to candidate dies in blocks of size 50 to balance efficiency and accuracy (line 6-11). For each block, the algorithm selects the optimal die using our cost model, which considers DRAM access, computation, and die-to-die communication. Finally, we merge blocks allocated to the same die to generate the final allocation plan (line 12).

Data-Driven Predictor: Our predictor algorithm, implemented by the global CP, uses current MoE kernel expert selection information to predict popular experts for the next token on

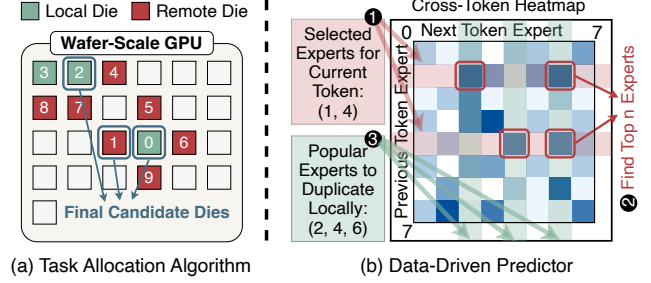


Figure 11. Proposed task allocation algorithm and data-driven predictor.

each die. This prediction result is transferred to local CPs and configured in each die’s PDU to guide hardware-managed local HBM. As shown by the red shadow in Figure 11(b), we first identify corresponding rows from the heatmap based on current expert selection (❶), then select the top *n* experts from each row (❷) and identify corresponding experts for the next token as prediction results, denoted by the green shadow (❸). In this example, a die computes experts 1 and 4 during the current MoE kernel and we predict experts 2, 4, and 6 will be used next. Since this die only reads experts 1 and 4 currently, we duplicate only expert 4 in its local DRAM.

V. EVALUATION

A. Experiment Setup

Methodology: We conduct experiments using event-driven simulation on a validated simulator. Expert selection traces are collected by deploying SGLang [51] on an 8×H100 DGX server and an 8×H200 AWS instance.

We developed a custom multi-chiplet GPU simulator in Python, as existing tools are inadequate for our needs. Cycle-accurate simulators such as Gem5 [52], gpgpusim [53], and mgpusim [54] accurately model single GPUs but are prohibitively slow for large-scale systems with 20+ dies and batch sizes exceeding 15,000. Event-driven simulators such as ASTRA-sim [55] support multi-GPU systems but lack detailed microarchitecture modeling and do not support the single-GPU-like programming model we adopt. Our simulator models all key multi-chiplet GPU components, including LLC, HBM, compute units, and D2D links across all dies, with a central resource manager that captures contention and congestion. We validated the simulator against real measurements from an 8×H100 DGX server, as detailed in subsection V-B.

Metric: We measure the throughput of MoE layers during the decode stage as modern LLM serving systems show a trend toward fine-granularity disaggregation. Traditional LLMs benefit from separating prefill and decode stages across different machines, as demonstrated by DistServe [56] and subsequent works [57], [58]. For MoE models, this disaggregation extends further. MegaScale-Infer [15] separates attention and MoE operations onto different machines for optimal batch sizes. Following this trend, we focus on optimizing MoE operations during the decode stage.

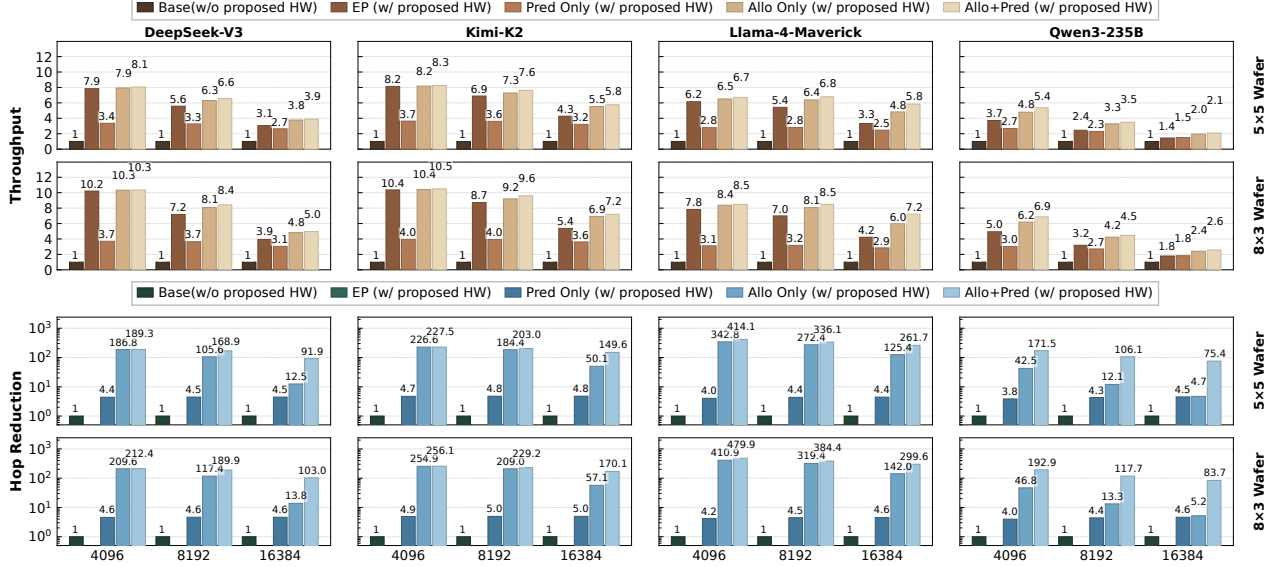


Figure 12. Throughput of MoE layers (Top) and hop number reduction ratio (Bottom). All figures are scaled to baseline.

Table I
HARDWARE CONFIGURATIONS

	X-die	Y-die	DRAM BW	D2D BW	DRAM	Cmpt Power per Die (FP16)
Dojo	5	5	3.35 TB/s	1.7 TB/s	80GB	989 TFLOPS
TSMC-SoW	3	8	3.35 TB/s	1.7 TB/s	80GB	989 TFLOPS
Dojo-Enhanced	5	5	8 TB/s	2 TB/s	180GB	4500 TFLOPS
Other Params	LLC hit latency: 100ns, LLC miss penalty: 110ns, LLC write latency: 30ns, LLC size: 64 MB D2D link latency: 200ns, Routing Alg: XY routing, Command and address size for each remote request: 16B Loca HBM access latency: 300 ns					

Hardware Configuration: We evaluate two multi-chiplet topologies: Tesla Dojo [59], [60] and the TSMC SoW roadmap [61]. As summarized in Table I, Dojo uses a 5×5 2D mesh, while TSMC SoW adopts an 8×3 2D mesh. These choices reflect a deployed system (Dojo) and near-future industry support (TSMC SoW).

For both the Dojo and TSMC SoW configurations, each chiplet is H100-like, providing 1,000 TFLOPS FP16 compute, 80GB HBM, 3.35TB/s local HBM bandwidth, and 1.7 TB/s inter-die bandwidth to adjacent chiplets. We also include an extended experiment in subsection V-F with a Dojo-Enhanced configuration, where each die is B300-like to reflect an anticipated hardware performance trend in the future. We reserve 10% of DRAM for system and hardware management.

Baseline Configurations: We compare our approach against the simple strategy currently used by GPU.

The **Base** configuration adopts an EP-like data placement and assigns an equal number of experts to each die. However, the entire wafer operates as a single large GPU: each die handles the same amount of expert computation without considering expert placement.

EP assigns each expert’s computation to the die where it resides, as also adopted by MoEntwine [45]. This eliminates all D2D communication but can cause severe workload imbalance. Note that even under EP, our Global CP and Local CP architecture remains necessary, as expert placement information is still required.

We implement three variants: **Allo Only** uses solely our task allocation strategy; **Pred Only** includes only the data-driven predictor; and **Allo+Pred** combines both techniques. These configurations evaluate the individual and combined effects of our proposed methods.

Models and Workloads: We conduct evaluations with real traces collected from Qwen3 and Deepseek V3. The traces are gathered from diverse datasets, including MMLU [35], MMLU Pro [36], ChineseSimpleQA [62], and LiveCodeBench [63], comprising over 24,000 requests per model. Each test batch is filled by sequentially adding requests in the order of MMLU, MMLU-Pro (CH), ChineseSimpleQA, and LiveCodeBench until the target batch size is reached.

B. Validation of Simulator

We validate our simulator using real measurements from an 8×H100 DGX server. We evaluate both single-GPU execution and two-GPU peer-to-peer (P2P) communication.

For single-GPU execution, we benchmark one expert in a MoE layer, which consists of three GEMM operations, across varying batch sizes for both DeepSeek and Qwen.

For P2P communication, we measure data migration between two GPUs over payload sizes ranging from 4 KB to 4 GB. To ensure simulation fidelity, we calibrate key parameters to fit the measured data. As shown in Figure 13, the simulator’s error remains within 5% for all test cases.

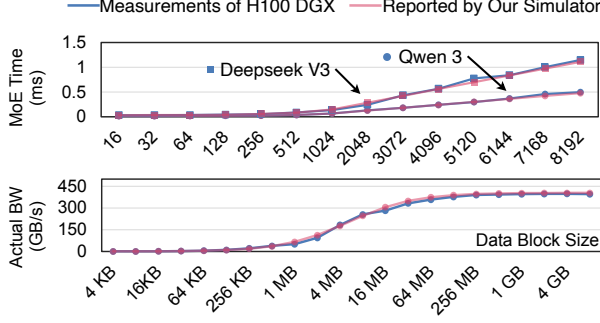


Figure 13. Simulator validation with real data generated from 8xH100 DGX, including both MoE Layer (Top) and P2P data transfer (Bottom) test cases.

C. Throughput

We evaluate MoE decode stage throughput in Figure 12, with results normalized to the baseline configuration.

Comparison across models: Our Allo+Pred strategy achieves $7.0\times$, $8.2\times$, $7.3\times$, and $4.1\times$ throughput improvement on Deepseek, Kimi, Llama, and Qwen, respectively. Deepseek and Kimi benefit more due to their larger expert count (256 vs. 128) and more complex selection patterns.

Comparison across chiplet architectures: Our strategy shows $6.0\times$ improvement on Dojo and $7.5\times$ on TSMC, despite similar die counts (25 vs. 24). TSMC’s rectangular layout places dies farther apart, introducing more inter-unit communication without strategic task allocation, hence the larger gain under our strategy.

Comparison with EP: At small batch sizes such as 4096, our strategy and EP perform similarly: few tokens per expert make execution memory-bound, so splitting one expert across multiple dies offers no benefit, and our strategy degenerates to EP. The advantage emerges at larger batches, achieving $1.44\times$ speedup over EP at batch size 16,384.

D. Hop Reduction

We report hop counts in Figure 12 to show the reduction in inter-unit communication. Hop count is the sum of Manhattan distances for all cross-unit communications. Higher hop counts indicate frequent cross-die data movement. We normalize results to baseline and report hop reduction ratios, where a ratio of 10 means the hop count is reduced to 1/10.

Pred Only reduces hop counts by $4.5\times$, aligning with performance improvement of $3.0\times$. This indicates cross-unit communication is the primary bottleneck in baseline, and reducing hop counts proportionally improves performance.

Allo Only reduces hop counts by $142\times$, exceeding the performance improvement of $6.3\times$. This shows that with our allocation algorithm, inter-unit communication is no longer the sole bottleneck. While reducing hop counts still improves performance, the improvement is not proportional.

Allo+Pred reduces hop counts by over $213\times$ compared to baseline. However, performance improvement is only $6.63\times$ over baseline, with just $1.1\times$ average improvement over Allo

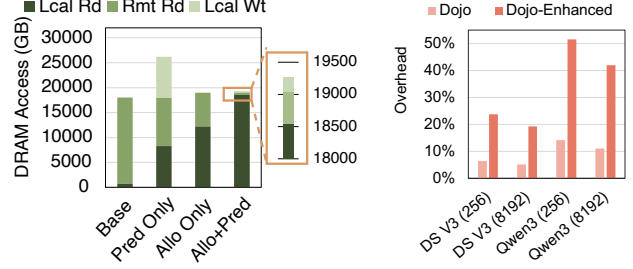


Figure 14. DRAM access breakdown for Qwen3 on TSMC-SoW Configuration with batch size 4096.

Figure 15. Host CPU implementation overhead under varying models and batch sizes.

Only. This demonstrates that hop count is no longer a performance bottleneck. With the help of our task allocation algorithm, most tasks are distributed to local dies holding related experts, with only extremely popular experts requiring remote allocation. This leads to minimal D2D traffic and shifts the bottleneck to workload distribution.

E. DRAM Access Breakdown

We provide a breakdown of DRAM access patterns in Figure 14 to show how our strategies reduce inter-unit communication. We categorize DRAM access into three types: reads from local dies, reads from remote dies, and writes to local dies, where writes to local dies only occur when we duplicate a remote expert locally. Most reads in the baseline are from remote dies, resulting in high inter-unit traffic and poor performance. With our strategies (Pred Only, Allo Only, and Allo+Pred), most remote DRAM reads are converted to local DRAM reads, significantly reducing traffic. Compared with Pred Only, Allo+Pred achieves fewer remote reads by allocating most tasks to local dies, with only extremely popular experts requiring computation across multiple dies. Compared with Allo Only, Allo+Pred further reduces remote reads by caching popular experts in local HBM.

F. Comparison with Host CPU-Based Implementation

Our task allocation algorithm runs on a new GPU command processor, but it could, in principle, be executed on the host CPU with a higher overhead. As shown in Figure 15, we evaluate both the Dojo and Dojo-Enhanced configurations. In Dojo, the overhead of host-CPU allocation is 5.2%–6.4% for DeepSeek V3 and 11.1%–14.2% for Qwen3. In Dojo-Enhanced, the overhead rises to 19.3%–23.8% for DeepSeek V3 and 42.0%–51.6% for Qwen3.

DeepSeek vs Qwen: Qwen3 incurs higher overhead than DeepSeek V3 due to CPU-GPU data transfers over PCIe, which occur once per MoE layer. The CPU needs the Expert Distribution Table from the GPU to run the allocator, and the allocation results must be sent back to the GPU before kernel execution. Qwen3 has (i) more MoE layers (94 vs. 58), increasing transfer frequency, and (ii) smaller per-layer compute, which amplifies the relative cost of transfers.

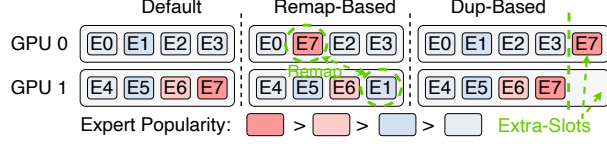


Figure 16. Demonstration of expert placement strategies.

Dojo vs Dojo-Enhanced: Dojo-Enhanced shows over 3.7 $\times$ higher overhead than Dojo because its GPU dies are significantly faster, making fixed PCIe transfer costs dominate more. As GPU performance outpaces interconnect bandwidth, implementing the allocator in the GPU command processor becomes increasingly necessary to sustain performance.

G. Area and Power Overhead

We estimate the area and power overhead of all added modules in Table II. Our design supports up to 100 layers with 512 experts per layer, well beyond SOTA MoE model (Kimi-K2: 61 layers, 384 experts). The full heatmap (50 MB) is stored in Global CP DRAM, with a 0.5 MB on-chip cache buffering one layer at a time. The Prediction Table is implemented in registers due to its small size; all other components use SRAM. Registers are synthesized with Yosys [64] and SRAM is modeled with CACTI [65], both scaled to 5 nm to match the H100 process node. Global and Local CP area estimates are derived from ARM core data. As shown in Table II, total area and power overhead is less than 0.04%.

Table II
AREA AND POWER OVERHEAD.

	Capacity	Bit Width	Num Per Wafer	Tot Area (mm ²)	Tot Power (mW)
Prediction Table	128 B	16 bit	25	0.0020	55.75
Address Translation Unit	4.25 KB	68 bit	25	0.0048	334.25
Local CP (A72) [66]	N/A	N/A	25	~ 7.5000	~ 7000
Expert Distribution Table	4.5 KB	72 bit	1	0.0002	13.94
Heatmap Cache	0.5 MB	512 bit	1	0.0278	184.67
Global CP (A76) [67]	N/A	N/A	1	~ 1.1000	~ 1000
Total				6.13	8588.61
Overhead (25-die wafer)				$\sim 0.04\%$	$\sim 0.04\%$

VI. CASE STUDY 2: PREFILL-GUIDED DECODE EXPERT PLACEMENT ON REAL GPU CLUSTERS

A. Introduction

Workload imbalance is one of the biggest challenges in large-scale MoE serving (200+ GPUs). EPLB [68] addresses this by dynamically adjusting expert placement, but it is triggered every 3000+ steps and relies on periodically collected profiling data [69]. A natural question then arises: how to set expert placement for the initial ~ 1000 decode tokens when no profiling data are yet available? This is especially pressing for short-output requests, for which EPLB never collects enough data to be effective. Inspired by [Insight 1](#), which reveals temporal correlation between prefill and decode stages, we propose leveraging prefill-stage expert selection information to guide expert placement for initial decode steps.

Algorithm 2: Prefill-Guided Expert Placement

Input: Prefill traces $\mathcal{D}$, GPU count G , extra slots per GPU R
Output: Per-layer expert-to-GPU assignment $\{\mathcal{S}_g\}_{g=1}^G$
Notation: E : total experts; $f_{l,e}$: freq of expert e at layer l ; L_g : load of GPU g ; r_g : remaining slots on GPU g ; $\delta_{e,g}$: $\max_{g'} L_{g'}$ change after copying expert e to GPU g

```

1 Function remap_based_placement( $\mathcal{D}, G$ ):
2   for each layer  $l$  do
3     Compute  $f_{l,e}$  from  $\mathcal{D}$ ; sort exps by decreasing  $\text{Cost}(f_{l,e})$ ;
4      $L_g \leftarrow 0$  for all  $g$ ;
5     for each expert  $e$  in sorted order do
6       Assign  $e$  to least-loaded GPU  $g^*$  s.t.  $|\mathcal{S}_{g^*}| < E/G$ ;
7        $L_{g^*} += \text{Cost}(f_{l,e})$ ;
8   return  $\{\mathcal{S}_g\}$  for each layer;

9 Function dup_based_placement( $\mathcal{D}, G, R$ ):
10  for each layer  $l$  do
11    Compute  $f_{l,e}$  from  $\mathcal{D}$ ; generate default placement  $\mathcal{S}_g$ ;
12     $r_g \leftarrow R$  for all  $g$ ;
13     $L_g \leftarrow \sum_{e \in \mathcal{S}_g} \text{Cost}(f_{l,e})$  for all  $g$ ;
14    for  $i \leftarrow 1$  to  $R \cdot G$  do
15       $(e^*, g^*) \leftarrow \arg \min_{e,g: r_g > 0, g \notin \text{hosts}(e)} \delta_{e,g}$ ;
16      Assign  $e^*$  to  $\mathcal{S}_{g^*}$ ;  $r_{g^*} \leftarrow r_{g^*} - 1$ ;
17      update affected  $L_g$ ;
18  return  $\{\mathcal{S}_g\}$  for each layer;

```

As shown in Figure 16, we design two placement algorithms (details in algorithm 2). The *Remap-based* algorithm keeps the number of experts per GPU unchanged and reassigns experts across GPUs for a more balanced workload: it sorts experts by decreasing roofline cost and greedily assigns each to the least-loaded GPU, subject to a uniform capacity of E/G experts per GPU. The *Duplication-based* algorithm reserves extra expert slots on each GPU and uses prefill traces to duplicate hot experts, thereby avoiding congestion: starting from the default contiguous layout (e.g., experts 0–15 on GPU,0, 16–31 on GPU,1, etc.), it greedily adds up to R extra replicas per GPU, selecting at each step the (expert, GPU) pair that maximally reduces the bottleneck load $\max_g \text{load}_g$; tokens of a replicated expert are evenly split among all its copies. Both algorithms use a roofline-based cost model to estimate per-GPU load.

B. Methodology

We deploy Qwen3-235B with SGLang on 8 $\times$ H100 GPUs with NVLink. We build a distributed profiler by inserting `cuda.Event` timers into SGLang to measure individual operations (attention, top- k , all-to-all, and MoE) on each GPU independently. We manipulate expert placement through SGLang’s `init_expert_location` interface and use DeepEP as the MoE backend. The `ep_dispatch_algorithm` is set to “dynamic” so that tokens are evenly distributed across replicas of a duplicated expert.

Metric. We report MoE computation time, i.e., all three expert linear layers, excluding attention, all-to-all, and top- k .

Model and Benchmark. We evaluate on Qwen3-235B (94 MoE layers, 128 experts per layer, 8 selected). We use MMLU and Global-MMLU datasets, following the original ordering. Batch sizes range from 64 to 16,384.

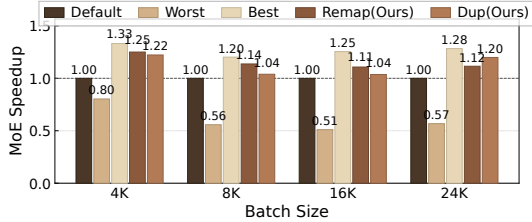


Figure 17. Performance of our prefill-aware expert placement.

Baselines. *Default* is the standard contiguous placement used by Qwen and SGLang (experts 0–15 on GPU-0, 16–31 on GPU-1, etc.). *Best* and *Worst* are the theoretically optimal and worst placements generated with oracle decode-stage selections (not available in practice). *Remap* and *Dup* are our two prefill-guided strategies. For *Dup*, we use one extra slot per GPU, yielding $128+8=136$ experts per layer.

C. Results

As shown in Figure 17, *Remap* and *Dup* achieve speedups of 15.5% and 12.5% over *Default*, respectively, and deliver over $2\times$ speedup compared with *Worst*. Both remain within 10% of *Best*, which exploits oracle decode-stage information unavailable in practice, which demonstrates the effectiveness of our approach. Since the two algorithms perform comparably, one can choose between them to fit different memory and system constraints.

We note that our 8-GPU EP scale inherently limits the achievable improvement: with EP8, each GPU holds 16 experts per layer, so every GPU likely contains a mix of hot and cold experts, naturally yielding a relatively balanced workload even under the *Default* layout (the max/min execution-time ratio is only about $1.3\times$). We expect greater speedups at larger EP scales where load imbalance is more pronounced.

VII. DISCUSSION

Both the wafer-scale GPU architecture and the prefill-guided expert placement strategy serve as case studies demonstrating the practical applicability of our profiling insights, which constitute the paper’s primary contribution. Specifically, the wafer-scale GPU design follows [Insight 3](#) for task allocation and leverages temporal relation insights ([Insight 1](#) and part of [Insight 2](#)) to build a data-driven predictor. The prefill-guided placement strategy utilizes [Insight 1](#) to guide decode-stage expert placement using information collected during prefill.

Importantly, our insights extend far beyond these two case studies and can benefit a wide range of MoE serving systems, including multi-GPU clusters (Multi-Node DGX [9], [15], [70] and NVL72 [71]), CXL/CPU-based memory disaggregation [12], [31], flash-based multi-tier systems [72], [73], PIM architectures [33], [74], [75], and other emerging platforms.

VIII. RELATED WORK

MoE model behavior studies: Several MoE model tech reports [76]–[79] provide the MoE routing patterns as part of

their evaluation. For example, the Mixtral report [77] shows the temporal locality of expert selection by reporting the percentage of repetitive assignment. The OLMoE report [76] shows the co-activation pattern and domain specialization among the experts. A blog post from SGLang [28] shows expert distribution statistics for the DeepSeekV3 model and the inherent imbalance in expert selection and similarity between prefill and decode. None of these studies provides a comprehensive profiling across multiple large-scale ($>200B$) MoE models, nor do they present a data-movement-centric methodology to highlight the opportunities like this paper.

Data movement optimization for MoE LLM inference

Various prior works [9], [16], [31], [33], [80]–[87] have focused on improving the efficiency of MoE LLM inference by reducing data movement. For example, Lina and SmartMoE [81], [88] exploit expert selection skewness to dynamically schedule resources during inference and balance traffic across GPUs. LYNX [82] dynamically reduces active experts while preserving model accuracy. Pre-gate MoE [80] uses a pre-gating function to alleviate the dynamic nature of expert selection. Sida [85] builds an offline hash function to predict expert usage and reduce data movement between CPU and GPU. MoE-Lightning [31] leverages a CPU-GPU pipeline and paged weights to improve resource utilization. Eliseev and Mazur [84] exploit expert locality and leverage LRU caching to manage GPU and CPU memory. This work targets data movement reduction in MoE LLM inference. Our cross-model profiling reveals optimization principles that apply broadly to current and future systems regardless of scale.

Wafer-Scale and Chiplet Architectures As single-chip scaling slows, wafer-scale and chiplet packaging offer promising paths for improved compute efficiency. Prior work targets either interconnect design [89]–[95] or data placement for specific algorithms [25], [96] and applications [26], [97], [98]. In contrast, we are the first to study MoE LLM serving on wafer-scale GPUs and propose data-movement-centric HW/SW co-design optimizations.

IX. CONCLUSION

Unlike prior MoE serving studies that take system-centric approaches with deployment-specific strategies, we study MoE serving from a model-focused perspective. We conduct comprehensive data-movement-centric profiling of state-of-the-art MoE models (200B–1000B) to extract system-agnostic insights, revealing structured patterns underlying seemingly random data movement and providing actionable guidance on system design. We validate these insights on both a future wafer-scale GPU architecture and existing multi-GPU systems, achieving significant performance improvements through minimal architectural modifications and lightweight software design, demonstrating their broad applicability.

ACKNOWLEDGMENT

We thank all reviewers for their constructive feedback and insightful suggestions. This work is partially supported by Samsung Semiconductor.

A.1 Abstract

This artifact packages the code, traces, scripts, and plotting utilities for reproducing the paper’s main results across the two case studies.

Case Study 1 is a CPU-runnable wafer-scale GPU simulator for MoE inference. It evaluates our expert allocation and prediction strategies across four large-scale MoE models and two chiplet topologies, reproducing Figure 12.

Case Study 2 contains the real-GPU expert placement experiments and reproduces Figure 17 on an 8×H100 system. It requires a specialized GPU software stack. Both artifacts provide a `main_ae.py` workflow for downloading traces, running experiments, and generating figures.

A.2 Artifact Check-List

Program: Python 3.

Run-time environment: Linux with Python ≥ 3.10 .

Hardware: Case Study 1 requires a CPU server with ≥ 64 GB RAM. Case Study 2 requires an 8×NVIDIA H100 80 GB GPU server.

Output: Paper figures and CSV result files.

Disk space: About 80 GB for one model and up to 300 GB for all four models.

Experiment time: Case Study 1 takes 8–12 hours for one model or 18–36 hours for all models. Case Study 2 takes 12–16 hours.

Publicly available: Yes.

Code license: Apache-2.0.

A.3 Description

A.3.1 How to Access: Case Study 1 (wafer-scale GPU simulator): GitHub: `waferscale_gpu_moe_sim`; DOI: 10.5281/zenodo.19617713.

Case Study 2 (real-GPU expert placement): GitHub: `moe_exp_placement`; DOI: 10.5281/zenodo.19617695.

Each repository contains a `README.md` with setup, execution, and troubleshooting instructions. The Zenodo archives provide persistent snapshots of the evaluated artifact versions.

A.3.2 Hardware Dependencies: Case Study 1 runs on a CPU server with at least 64 GB RAM and does not require a GPU. Case Study 2 requires an 8×NVIDIA H100 80 GB GPU server, CUDA 12.0 or newer, and about 300 GB of disk space. Reviewers without GPU access can still evaluate the primary simulator artifact.

A.3.3 Software Dependencies: Case Study 1 requires Python ≥ 3.10 plus `numpy`, `pandas`, and `matplotlib`; the scripts install them automatically. Case Study 2 additionally requires `PyTorch`, a modified `SGLang` fork, `DeepEP`, and `DeepGEMM`. The repository documents exact installation commands and environment settings.

A.3.4 Datasets: Both artifacts use pre-recorded MoE expert-selection traces from MMLU. The traces are hosted on HuggingFace and downloaded automatically by the AE scripts.

A.4 Installation

Installation instructions are provided in each repository’s `README.md`. Case Study 1 is self-contained and intended as the default AE path. Case Study 2 includes GPU stack and communication library setup.

A.5 Experiment Workflow

Both repositories provide a `main_ae.py` entry point. The script downloads traces, runs experiments, collects CSV files, and regenerates the corresponding paper figure. Reviewers may also run individual model configurations using the repository `README.md` commands.

A.6 Evaluation and Expected Results

Case Study 1 reproduces Figure 12. The simulator is deterministic, so generated results should match the reported trends when using the same traces and configuration files.

Case Study 2 reproduces Figure 17. Because it measures real GPU execution, small timing variations are expected from thermals, system load, NCCL non-determinism, and SGLang micro-batching. In our runs, variation is typically within $\pm 5\%$. The high-level result is stable: prefill-aware placement improves MoE kernel performance by about 5–25% over the default placement.

A.7 Methodology

Submission, reviewing, and badging follow the ACM artifact review guidelines and the cTuning AE guidelines.

REFERENCES

- [1] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers, “Using an llm to help with code understanding,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ser. ICSE ’24, 2024.
- [2] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, “Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” *arXiv*, 2021.
- [3] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv*, 2023.
- [4] Y. Lu, W. Zhu, L. Li, Y. Qiao, and F. Yuan, “Llamax: Scaling linguistic horizons of llm by enhancing translation capabilities beyond 100 languages,” *arXiv*, 2024.
- [5] S. K. Dam, C. S. Hong, Y. Qiao, and C. Zhang, “A complete survey on llm-based ai chatbots,” *arXiv*, 2024.
- [6] S. Vakayil, D. S. Juliet, S. Vakayil *et al.*, “Rag-based llm chatbot using llama-2,” in *2024 7th International Conference on Devices, Circuits and Systems (ICDCS)*, 2024.
- [7] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, B. Zhu, H. Zhang, M. Jordan, J. E. Gonzalez *et al.*, “Chatbot arena: An open platform for evaluating llms by human preference,” in *Forty-first International Conference on Machine Learning*, 2024.
- [8] S. Go and D. Mahajan, “Moetuner: Optimized mixture of expert serving with balanced expert placement and token routing,” *arXiv preprint arXiv:2502.06643*, 2025.
- [9] S. Zhang, N. Zheng, H. Lin, Z. Jiang, W. Bao, C. Jiang, Q. Hou, W. Cui, S. Zheng, L.-W. Chang, Q. Chen, and X. Liu, “COMET: Fine-grained computation-communication overlapping for mixture-of-experts,” in *Eighth Conference on Machine Learning and Systems*, 2025. [Online]. Available: <https://openreview.net/forum?id=fGgQS5VW09>
- [10] DeepSeek-AI, “DeepSeek-V4: Towards highly efficient million-token context intelligence,” Technical report, 2026. [Online]. Available: <https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash>

- [11] GLM-5 Team, A. Zeng, X. Lv, Z. Hou, Z. Du, Q. Zheng, B. Chen, D. Yin *et al.*, “GLM-5: from vibe coding to agentic engineering,” *arXiv preprint arXiv:2602.15763*, 2026.
- [12] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, “Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025.
- [13] S. Tairin, S. Mahmud, H. Shen, and A. Iyer, “emoe: Task-aware memory efficient mixture-of-experts-based (moe) model inference,” *arXiv preprint arXiv:2503.06823*, 2025.
- [14] J. Yao, Q. Anthony, A. Shafi, H. Subramoni, and D. K. D. Panda, “Exploiting inter-layer expert affinity for accelerating mixture-of-experts model inference,” in *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2024.
- [15] R. Zhu, Z. Jiang, C. Jin, P. Wu, C. A. Stuardo, D. Wang, X. Zhang, H. Zhou, H. Wei, Y. Cheng *et al.*, “Megascall-infer: Serving mixture-of-experts at scale with disaggregated expert parallelism,” *arXiv preprint arXiv:2504.02263*, 2025.
- [16] A. Skliar, T. van Rozendaal, R. Lepert, T. Boinovski, M. Van Baalen, M. Nagel, P. Whatmough, and B. E. Bejnordi, “Mixture of cache-conditional experts for efficient mobile device inference,” *arXiv preprint arXiv:2412.00099*, 2024.
- [17] K. T. Chitty-Venkata, S. Madireddy, M. Emani, and V. Vishwanath, “Lexi: Layer-adaptive active experts for efficient moe model inference,” *arXiv preprint arXiv:2509.02753*, 2025.
- [18] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, “Deepseek-v3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [19] Meta. (2025) Llama4 technical report. [Online]. Available: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>
- [20] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [21] K. Team, Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen *et al.*, “Kimi k2: Open agentic intelligence,” *arXiv preprint arXiv:2507.20534*, 2025.
- [22] A. Smith, G. H. Loh, J. Wu, S. Naffziger, T. Huang, H. McIntyre, R. Mangaser, W. Jung, and R. Swaminathan, “Amd instinct™ mi300x accelerator: Packaging and architecture co-optimization,” in *2024 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. IEEE, 2024.
- [23] P. Dalmia, R. S. Kumar, and M. D. Sinclair, “Cpelide: Efficient multi-chiplet gpu implicit synchronization,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.
- [24] NVIDIA, “Nvidia blackwell architecture overview,” <https://resources.nvidia.com/en-us-blackwell-architecture>, 2025.
- [25] C. He, Y. Huang, P. Mu, Z. Miao, J. Xue, L. Ma, F. Yang, and L. Mai, “Waferllm: Large language model inference at wafer scale,” in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. USENIX Association, 2025.
- [26] Z. Xu, D. Kong, J. Liu, J. Li, J. Hou, X. Dai, C. Li, S. Wei, Y. Hu, and S. Yin, “Wsc-llm: Efficient llm service and architecture co-exploration for wafer-scale chips,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1–17.
- [27] S. T. from LMSYS Org. (2025) Together with sglang: Best practices for serving deepseek-r1 on h20-96g. [Online]. Available: <https://lmsys.org/blog/2025-09-26-sglang-ant-group/>
- [28] —. (2025) Deploying deepseek with pd disaggregation and large-scale expert parallelism on 96 h100 gpus. [Online]. Available: <https://lmsys.org/blog/2025-05-05-large-scale-ep/>
- [29] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, 2017.
- [30] S. Masoudnia and R. Ebrahimpour, “Mixture of experts: a literature survey,” *Artificial Intelligence Review*, pp. 275–293, 2014.
- [31] S. Cao, S. Liu, T. Griggs, P. Schaffhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, “Moe-lightning: High-throughput moe inference on memory-constrained gpus,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.
- [32] J. Suo, X. Liao, L. Xiao, L. Ruan, J. Wang, X. Su, and Z. Huo, “Coserve: Efficient collaboration-of-experts (coe) model inference with limited memory,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.
- [33] S. Yun, K. Kyung, J. Cho, J. Choi, J. Kim, B. Kim, S. Lee, K. Sohn, and J. H. Ahn, “Duplex: A device for large language models with mixture of experts, grouped query attention, and continuous batching,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.
- [34] P. Schober, C. Boer, and L. A. Schwarte, “Correlation coefficients: appropriate use and interpretation,” *Anesthesia & analgesia*, 2018.
- [35] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” *arXiv preprint arXiv:2009.03300*, 2020.
- [36] Y. Wang, X. Ma, G. Zhang, Y. Ni, A. Chandra, S. Guo, W. Ren, A. Arulraj, X. He, Z. Jiang *et al.*, “Mmlu-pro: A more robust and challenging multi-task language understanding benchmark,” *Advances in Neural Information Processing Systems*, 2024.
- [37] M. Lundstrom, “Moore’s law forever?” *Science*, 2003.
- [38] Y.-C. Hu, Y.-M. Liang, H.-P. Hu, C.-Y. Tan, C.-T. Shen, C.-H. Lee, and S. Hou, “Cowos architecture evolution for next generation hpc on 2.5 d system in package,” in *2023 IEEE 73rd Electronic Components and Technology Conference (ECTC)*, 2023.
- [39] J. Shin, H. Eslampour, S. Jeong, W. Kim, S. Yong, S.-O. Ahn, E. Park, and S. Song, “Signal integrity of die-to-die interface with advanced packages for co-packaged optics,” in *2024 IEEE 33rd Conference on Electrical Performance of Electronic Packaging and Systems (EPEPS)*, 2024.
- [40] R. Mahajan, R. Sankman, N. Patel, D.-W. Kim, K. Aygun, Z. Qian, Y. Mekonnen, I. Salama, S. Sharan, D. Iyengar *et al.*, “Embedded multi-die interconnect bridge (emib)—a high density, high bandwidth packaging interconnect,” in *2016 IEEE 66th Electronic Components and Technology Conference (ECTC)*, 2016.
- [41] NVIDIA, “Nvidia gtc 2025,” <https://www.nvidia.com/gtc/>, 2025.
- [42] S. Hou, W. C. Chen, C. Hu, C. Chiu, K. Ting, T. Lin, W. Wei, W. Chiou, V. J. Lin, V. C. Chang *et al.*, “Wafer-level integration of an advanced logic-memory system through the second-generation cowos technology,” *IEEE Transactions on Electron Devices*, 2017.
- [43] P.-C. Shih, A.-J. Su, K.-H. Tam, T.-C. Huang, K. Chuang, and J. Yeh, “Sow-x: A novel system-on-wafer technology for next generation ai server application,” in *2025 IEEE 75th Electronic Components and Technology Conference (ECTC)*. IEEE, 2025.
- [44] S. Hou, H. Hsia, C. Tsai, K. Ting, T. Yu, Y. Lee, F. Chen, W. Chiou, C. Wang, C. Wu *et al.*, “Integrated deep trench capacitor in si interposer for cowos heterogeneous integration,” in *2019 IEEE International Electron Devices Meeting (IEDM)*. IEEE, 2019, pp. 19–5.
- [45] X. Tang, J. Hou, D. Jiang, T. Wei, J. Liu, J. Deng, H. Wang, Q. Yang, H. Shang, C. Li *et al.*, “Moentwine: Unleashing the potential of wafer-scale chips for large-scale expert parallel inference,” *arXiv preprint arXiv:2510.25258*, 2025.
- [46] M. Joshi, B. Joo, Z. Susskind, A. Hendriksen, and K. Clark. (2026) Accelerating data processing with NVIDIA multi-instance GPU and NUMA node localization. NVIDIA Technical Blog. [Online]. Available: <https://developer.nvidia.com/blog/accelerating-data-processing-with-nvidia-multi-instance-gpu-and-numa-node-localization/>
- [47] D. Xu, Y. Li, Y. Sun, J. Ren, and Y. Sun, “Hdpat: Hierarchical distributed page address translation for wafer-scale gpus,” in *2026 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2026.
- [48] Z. Huang, S. Fan, C. Tang, X. Lin, S. Deng, and Y. Liu, “Hecaton: Training large language models with scalable chiplet systems,” *arXiv preprint arXiv:2407.05784*, 2024.
- [49] M. LeBeane, K. Hamidouche, B. Benton, M. Breternitz, S. K. Reinhardt, and L. K. John, “Comp-net: Command processor networking for efficient intra-kernel communications on gpus,” in *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques*, 2018.
- [50] A. Gutierrez, B. M. Beckmann, A. Dutu, J. Gross, M. LeBeane, J. Kalamatianos, O. Kayiran, M. Poremba, B. Potter, S. Puthoor *et al.*, “Lost in abstraction: Pitfalls of analyzing gpus at the intermediate language level,” in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2018.
- [51] L. Zheng, L. Yin, Z. Xie, C. L. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, “Sglang: Efficient execution of structured language model programs,” *Advances in neural information processing systems*, 2024.

- [52] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, “The gem5 simulator,” *ACM SIGARCH computer architecture news*, 2011.
- [53] A. Bakhoda, G. L. Yuan, W. W. Fung, H. Wong, and T. M. Aamodt, “Analyzing cuda workloads using a detailed gpu simulator,” in *2009 IEEE international symposium on performance analysis of systems and software*, 2009.
- [54] Y. Sun, T. Baruah, S. A. Mojumder, S. Dong, X. Gong, S. Treadway, Y. Bao, S. Hance, C. McCardwell, V. Zhao *et al.*, “Mgpusim: Enabling multi-gpu performance modeling and optimization,” in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019.
- [55] W. Won, T. Heo, S. Rashidi, S. Sridharan, S. Srinivasan, and T. Krishna, “Astra-sim2.0: Modeling hierarchical networks and disaggregated systems for large-model training at scale,” in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2023.
- [56] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “{DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.
- [57] B. Wu, S. Liu, Y. Zhong, P. Sun, X. Liu, and X. Jin, “Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism,” in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 640–654.
- [58] R. Qin, Z. Li, W. He, J. Cui, F. Ren, M. Zhang, Y. Wu, W. Zheng, and X. Xu, “Mooncake: Trading more storage for less computation—a {KVCACHE-centric} architecture for serving {LLM} chatbot,” in *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025.
- [59] E. Talpes, D. D. Sarma, D. Williams, S. Arora, T. Kunjan, B. Floering, A. Jalote, C. Hsiong, C. Poorna, V. Samant *et al.*, “The microarchitecture of dojo, tesla’s exa-scale computer,” *IEEE Micro*, 2023.
- [60] E. Talpes, D. Williams, and D. D. Sarma, “Dojo: The microarchitecture of tesla’s exa-scale computer,” in *2022 IEEE Hot Chips 34 Symposium (HCS)*, 2022.
- [61] TSMC. (2025) Tsmc’s next generation of system-on-wafer package will make today’s cpus and gpus look pathetically feeble in comparison. [Online]. Available: <https://www.pcgamer.com/hardware/processors/tsmcs-next-generation-of-system-on-wafer-packaging-will-make-todays-cpus-and-gpus-look-pathetically-feeble-in-comparison/>
- [62] Y. He, S. Li, J. Liu, Y. Tan, W. Wang, H. Huang, X. Bu, H. Guo, C. Hu, B. Zheng *et al.*, “Chinese simpleqa: A chinese factuality evaluation for large language models,” *arXiv preprint arXiv:2411.07140*, 2024.
- [63] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica, “Livecodebench: Holistic and contamination free evaluation of large language models for code,” *arXiv preprint arXiv:2403.07974*, 2024.
- [64] C. Wolf, J. Glaser, and J. Kepler, “Yosys-a free verilog synthesis suite,” in *Proceedings of the 21st Austrian Workshop on Microelectronics (Austrochip)*, vol. 97, 2013, pp. 1–6.
- [65] R. Balasubramanian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, “Cacti 7: New tools for interconnect exploration in innovative off-chip memories,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 14, no. 2, pp. 1–25, 2017.
- [66] G. Shan, Y. Zheng, C. Xing, D. Chen, G. Li, and Y. Yang, “Architecture of computing system based on chiplet,” *Micromachines*, vol. 13, no. 2, p. 205, 2022.
- [67] K. Wang, J. Chen, Y. Xu, Z. Yu, W. He, D. Tang, N. Sun, and Y. Bao, “Xiangshan: An open-source project for high-performance risc-v processors meeting industrial-grade standards,” *IEEE Micro*, 2025.
- [68] deepseek-ai, “Eplb: Expert parallelism load balancer,” 2026, gitHub repository (Python implementation: eplb.py). [Online]. Available: <https://github.com/deepseek-ai/EPLB>
- [69] vLLM Team. (2025) Expert parallel deployment — configuration. VLLM Documentation, version 0.12.0. [Online]. Available: https://docs.vllm.ai/en/v0.12.0/serving/expert_parallel_deployment/#configuration_1
- [70] C. Chen, M. Li, Z. Wu, D. Yu, and C. Yang, “Ta-moe: Topology-aware large scale mixture-of-expert training,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 22 173–22 186, 2022.
- [71] W. Li, J. Peng, Z. Jing, T. Zhang, Z. Long, X. Qiao, X. Chen, D. Yang, K. Duan, and J. Yang, “Dwdp: Distributed weight data parallelism for high-performance llm inference on nv172,” *arXiv preprint arXiv:2604.01621*, 2026.
- [72] Z. Yu, S. Liang, T. Ma, Y. Cai, Z. Nan, D. Huang, X. Song, Y. Hao, J. Zhang, T. Zhi *et al.*, “Cambricon-llm: A chiplet-based hybrid architecture for on-device inference of 70b llm,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2024.
- [73] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single gpu,” in *International Conference on Machine Learning*, 2023.
- [74] Y. Pan, Z. Xia, P.-K. Hsu, L. Hu, H. Kim, J. Sharda, M. Zhou, N. S. Kim, S. Yu, T. Rosing *et al.*, “Stratum: System-hardware co-design with tiered monolithic 3d-stackable dram for efficient moe serving,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 1–17.
- [75] Z. Yu, H. Ye, C. Zhou, O. R. Venkatachalam, Z. Pan, Z. Hu, J. Kim, W. W. Ro, P.-A. Tsai, S. Pei, Y. Kang, and Y. Ding, “Amma: A multi-chiplet memory-centric architecture for low-latency 1m context attention serving,” *arXiv preprint arXiv:2604.26103*, 2026.
- [76] N. Muennighoff, L. Soldaini, D. Groeneveld, K. Lo, J. Morrison, S. Min, W. Shi, P. Walsh, O. Tafjord, N. Lambert *et al.*, “Olmoe: Open mixture-of-experts language models,” *arXiv preprint arXiv:2409.02060*, 2024.
- [77] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, “Mixtral of experts,” *arXiv preprint arXiv:2401.04088*, 2024.
- [78] D. Dai, C. Deng, C. Zhao, R. Xu, H. Gao, D. Chen, J. Li, W. Zeng, X. Yu, Y. Wu *et al.*, “Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models,” *arXiv preprint arXiv:2401.06066*, 2024.
- [79] X. Sun, Y. Chen, Y. Huang, R. Xie, J. Zhu, K. Zhang, S. Li, Z. Yang, J. Han, X. Shu *et al.*, “Hunyuan-large: An open-source moe model with 52 billion activated parameters by tencent,” *arXiv preprint arXiv:2411.02265*, 2024.
- [80] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, “Pre-gated moe: An algorithm-system co-design for fast and scalable mixture-of-expert inference,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1018–1031.
- [81] J. Li, Y. Jiang, Y. Zhu, C. Wang, and H. Xu, “Accelerating distributed moe training and inference with lina,” in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 945–959.
- [82] V. Gupta, K. Sinha, A. Gavrilovska, and A. P. Iyer, “Lynx: Enabling efficient moe inference through dynamic batch-aware expert selection,” *arXiv preprint arXiv:2411.08982*, 2024.
- [83] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasicki, “Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=N5fVv6PZGz>
- [84] A. Eliseev and D. Mazur, “Fast inference of mixture-of-experts language models with offloading,” *arXiv preprint arXiv:2312.17238*, 2023.
- [85] Z. Du, S. Li, Y. Wu, X. Jiang, J. Sun, Q. Zheng, Y. Wu, A. Li, H. Li, and Y. Chen, “Sida: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models,” *Proceedings of Machine Learning and Systems*, vol. 6, pp. 224–238, 2024.
- [86] J. Li, S. Tripathi, L. Rastogi, Y. Lei, R. Pan, and Y. Xia, “Optimizing mixture-of-experts inference time combining model deployment and communication scheduling,” *arXiv preprint arXiv:2410.17043*, 2024.
- [87] C. Hwang, W. Cui, Y. Xiong, Z. Yang, Z. Liu, H. Hu, Z. Wang, R. Salas, J. Jose, P. Ram *et al.*, “Tutel: Adaptive mixture-of-experts at scale,” *Proceedings of Machine Learning and Systems*, vol. 5, pp. 269–287, 2023.
- [88] M. Zhai, J. He, Z. Ma, Z. Zong, R. Zhang, and J. Zhai, “{SmartMoE}: Efficiently training {Sparsely-Activated} models through combining offline and online parallelization,” in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*, 2023, pp. 961–975.
- [89] S. Rashidi, W. Won, S. Srinivasan, P. Gupta, and T. Krishna, “Fred: A wafer-scale fabric for 3d parallel dnn training,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 34–48.
- [90] Q. Yang, T. Wei, S. Guan, C. Li, H. Shang, J. Deng, H. Wang, C. Li, L. Wang, Y. Zhang *et al.*, “Pd constraint-aware physical/logical topology co-design for network on wafer,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 49–64.
- [91] X. Yu, D. Jiang, J. Deng, J. Liu, C. Li, S. Yin, and Y. Hu, “Cramming a data center into one cabinet, a co-exploration of computing and hardware

- architecture of waferscale chip,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 631–645.
- [92] Z. Li and D. Wentzlaff, “Lucie: A universal chiplet-interposer design framework for plug-and-play integration,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 423–436.
- [93] S. Chen, S. Pal, and R. Kumar, “Waferscale network switches,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 215–229.
- [94] Y. Feng, D. Xiang, and K. Ma, “Heterogeneous die-to-die interfaces: Enabling more flexible chiplet interconnection systems,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 930–943.
- [95] —, “A scalable methodology for designing efficient interconnection network of chiplets,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 1059–1071.
- [96] Z. Tan, H. Cai, R. Dong, and K. Ma, “Nn-baton: Dnn workload orchestration and chiplet granularity exploration for multichip accelerators,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 1013–1026.
- [97] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina *et al.*, “Simba: Scaling deep-learning inference with multi-chip-module-based architecture,” in *Proceedings of the 52nd annual IEEE/ACM international symposium on microarchitecture*, 2019, pp. 14–27.
- [98] M. Odema, L. Chen, H. Kwon, and M. A. Al Faruque, “Scar: Scheduling multi-model ai workloads on heterogeneous multi-chiplet module accelerators,” in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 565–579.